BANGLADESH UNIVERSITY OF ENGINEERING AND TECHNOLOGY
DEPARTMENT OF ELECTRICAL AND ELECTRONIC ENGINEERING

EEE 318 (July2025)

Control Systems I Laboratory

Final Project Report

Section: C2 Group: 01

Lane Following & Obstacle Avoidance Robot

Course Instructors:

Md. Jawad Ul Islam, Lecturer
Jonaidul Islam Sikder, Part-Time Lecturer

Signature of Instructor: _____

Academic Honesty Statement:

IMPORTANT! Please carefully read and sign the Academic Honesty Statement, below. Type the student ID and name, and put your signature. *You will not receive credit for this project experiment unless this statement is signed in the presence of your lab instructor.*

"In signing this statement, We hereby certify that the work on this project is our own and that we have not copied the work of any other students (past or present), and cited all relevant sources while completing this project. We understand that if we fail to honor this agreement, We will each receive a score of ZERO for this project and be subject to failure of this course."

Signature: _____ **Full Name: Kanto Banik** **Student ID: 2106164**	**Signature:** _____ **Full Name: Md. Rahat Mostafa** **Student ID: 2106166**
Signature: _____ **Full Name: Raihan Chowdhury Showrov** **Student ID: 2106168**	**Signature:** _____ **Full Name: Md. Nafshan Hussain** **Student ID: 2106170**
Signature: _____ **Full Name: Md. Maksudul Azom** **Student ID: 2106171**	**Signature:** _____ **Full Name: Asibur Rahman Rakib** **Student ID: 2106180**

Table of Contents

1 Abstract 1

2 Introduction 1

3 Design 2

3.1 Problem Formulation (PO(b)) 2

3.1.1 Identification of Scope 2

3.1.2 Literature Review 3

3.1.3 Formulation of Problem 4

3.1.4 Analysis 4

3.2 Design Method (PO(a)) 7

3.3 Circuit Diagram 9

3.5 Hardware Design 9

4 Implementation 12

4.1 Description 12

5 Design Analysis and Evaluation 14

5.1 Novelty 14

5.2 Design Considerations (PO(c)) 14

5.2.1 Considerations to public health and safety 14

5.2.2 Considerations to environment 14

5.2.3 Considerations to cultural and societal needs 14

5.3 Investigations (PO(d)) 15

5.3.1 Results and Analysis 15

5.4 Limitations of Tools (PO(e)) 16

6 Reflection on Individual and Team work (PO(i)) 17

6.1 Individual Contribution of Each Member 17

6.2 Mode of TeamWork 17

6.3 Diversity Statement of Team 17

6.4 Log Book of Project Implementation 18

7 Communication to External Stakeholders (PO(j)) 18

7.1 Executive Summary 18

7.2 User Manual 19

7.3 Github Link 20

7.4 YouTube Link 20

8 Project Management and Cost Analysis (PO(k)) 21
8.1 Bill of Materials 21
8.2 Calculation of Per Unit Cost of Prototype 22
9 Future Work (PO(l)) 22
10 References 24

1 Abstract

This project presents the design and implementation of a vision-based autonomous mobile robot capable of performing lane following and obstacle avoidance in a structured indoor environment. The system integrates computer vision, sensor-based perception, and closed-loop control to achieve reliable navigation. A monocular camera is used to capture real-time images, which are processed using advanced image processing techniques such as Gaussian filtering, Canny edge detection, and Hough Transform to extract lane features. The system is capable of detecting up to three lane lines (left, middle, and right) and dynamically estimating the lane center. A Proportional-Integral-Derivative (PID) controller is implemented to minimize the lateral deviation and ensure smooth trajectory tracking. In addition to lane following, ultrasonic sensors are used to detect obstacles in real time. A state-machine-based control strategy allows the robot to switch between different modes such as normal lane following, obstacle avoidance, counter-steering, and realignment. This ensures safe navigation while maintaining stability. Experimental results demonstrate that the system achieves reliable lane detection, smooth control performance, and effective obstacle avoidance. The proposed design provides a strong foundation for developing more advanced autonomous navigation systems.

2 Introduction

Autonomous navigation systems have become a key focus in modern engineering due to their wide applications in self-driving vehicles, robotics, and industrial automation. Traditional manual control systems are often prone to human error, which can result in inefficiencies and safety risks. Therefore, developing intelligent systems capable of independent navigation is of great importance.This project aims to design and implement a small-scale autonomous robot that can follow lane markings and avoid obstacles without human intervention. The system combines computer vision techniques, real-time sensor data processing, and control algorithms to achieve robust performance.

The primary objectives of this project are:

- To develop a vision-based lane detection system using image processing techniques
- To implement a PID-based control mechanism for smooth and stable motion
- To integrate ultrasonic sensors for obstacle detection and avoidance
- To design a decision-making system for autonomous navigation

The system demonstrates how multiple engineering concepts—including signal processing, embedded systems, and control theory—can be integrated into a single intelligent platform.

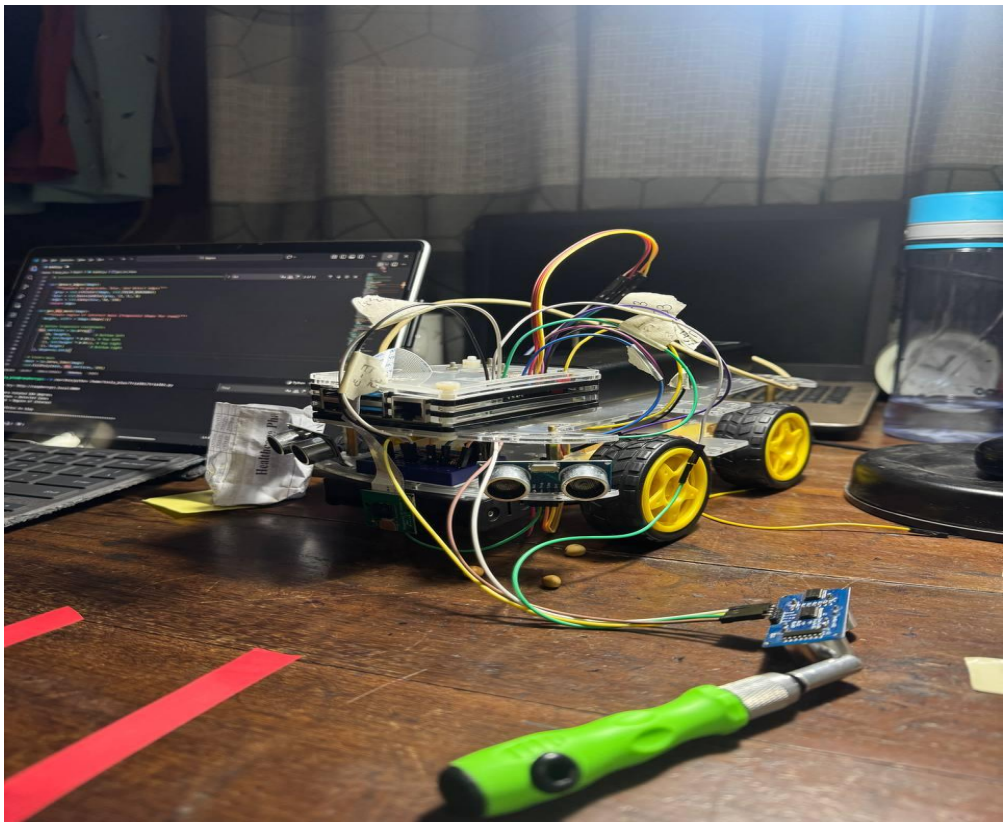


Figure: Overall System Concept

3 Design

3.1 Problem Formulation (PO(b))

3.1.1 Identification of Scope

The scope of this project defines the boundaries, capabilities, and operating conditions of the proposed lane-following and obstacle-avoidance robot system.

Operational Environment

- The system is designed for a controlled indoor environment.
- The floor contains clearly marked lane lines for detection.
- Lighting conditions are assumed to be moderately stable (not extreme dark or bright).

Functional Scope

The robot performs the following core functions:

- Detect lane lines using a monocular camera
- Follow the lane using real-time image processing
- Maintain alignment using a PID control system
- Detect obstacles using ultrasonic sensors
- Execute automatic obstacle avoidance and return to lane

Hardware Scope

The system includes:

- Camera module for vision
- Ultrasonic sensors (front, left, right)
- Differential drive motors
- Motor driver and microcontroller (e.g., Raspberry Pi)

Software Scope

- Image processing using OpenCV
- Control logic implemented in Python
- Real-time control using PID algorithm
- State-machine-based navigation system

3.1.2 Literature Review

Lane-following and autonomous navigation systems are commonly implemented using vision-based and sensor-based techniques. Classical methods such as Canny edge detection and Hough Transform are widely used for lane detection due to their simplicity and real-time performance. However, these methods are sensitive to lighting conditions and incomplete lane markings.

Recent approaches use deep learning models (CNNs) for lane detection, which provide higher accuracy and robustness but require high computational resources and large datasets.

For obstacle detection, systems typically use ultrasonic sensors, LiDAR, or depth cameras. Ultrasonic sensors are cost-effective but have limited range and accuracy.

Control of lane-following robots is commonly achieved using PID controllers, which provide stable and efficient real-time correction.

In this project, a combination of classical vision techniques, ultrasonic sensing, and PID control is used to achieve a balance between performance, cost, and computational efficiency.

3.1.3 Formulation of Problem

The development of an autonomous navigation system involves solving multiple interdependent problems under real-time constraints. The system must continuously perceive the environment, make decisions, and control motion accordingly.

Lane Detection Problem

The robot must accurately identify lane boundaries from camera input despite:

- Variations in lighting conditions
- Noise and distortions in captured images
- Partial or missing lane markings

Control Problem

The robot must maintain its position within the lane by minimizing lateral error. This requires:

- Continuous feedback from the vision system
- Real-time motor control adjustments
- Stability against oscillations and disturbances

Obstacle Avoidance Problem

The robot must detect and respond to obstacles by:

- Measuring distances using ultrasonic sensors
- Determining a safe avoidance direction
- Re-aligning with the lane after avoidance

3.1.4 Analysis

1. **Lane Detection (with ranges)**

Handled by: LaneDetector class

Preprocessing (**preprocess_frame**)

- **HSV Color Masking**
  - cv2.inRange()
    Typical Range:
  - H: 0–180, S: 50–255, V: 50–255
    (depends on lane color tuning)

- **Gaussian Blur**
  - cv2.GaussianBlur()
    BLUR_KERNEL_SIZE: 3–7 (odd numbers only)

- **Canny Edge Detection**
  - cv2.Canny()
    LOW Threshold: 50–150
    HIGH Threshold: 150–300

- **ROI Mask**
  - Bottom portion of frame
    ROI Height: ~50%–70% of image

Line Detection (detect_lines)

- Hough Transform
  - cv2.HoughLinesP()

Typical Ranges:

- Threshold: 20–100
- Min Line Length: 20–100 pixels
- Max Line Gap: 10–50 pixels

Clustering

- CLUSTER_X_THRESH

Range: 20–80 pixels

2. Control System (PD Controller with ranges)

Error Calculation

error = lane_center - frame_center

Range:

- Error: -160 to +160 (for 320px width)

Error Smoothing

filtered_error = α * raw_error + (1-α) * previous

SMOOTHING_FACTOR (α): 0.1 – 0.5

Motor Speed

left_speed = base_speed - output
right_speed = base_speed + output

Typical Range:

- Base Speed: 0.3 – 0.7
- Motor Speed: 0 to 1 (normalized)

3. Obstacle Detection (with ranges)

Handled by: Ultrasonic Sensors

Distance Measurement

Typical Range:

- Valid Distance: 2 cm – 400 cm

Decision Threshold

Obstacle condition:

- Front Distance < 15–30 cm → obstacle detected

Side Clearance

Used for turning:

- Safe distance: > 20–40 cm

4. Lane Change & State Machine (with ranges)

Avoidance States

- 'avoid_left' / 'avoid_right'

  Trigger:

- Front < 20 cm

Turning Sped Difference

Typical Speed difference: 0.2 – 0.5

Counter-Steer

Duration: 0.2 – 1.0 seconds

Realign

Speed: Slow forward: 0.2 – 0.4

Time: 1–2 seconds to reacquire lane

3.2 Design Method (PO(a))

The system design integrates multiple modules, including perception, control, and decision-making.

Vision-Based Lane Detection

The lane detection algorithm follows a structured pipeline:
Image Acquisition: The camera captures real-time frames of the environment.
Preprocessing :

- Conversion to grayscale and HSV color space
- Noise reduction using Gaussian blur

Edge Detection: The Canny edge detector is used to highlight lane boundaries.
Line Detection: The Hough Transform is applied to detect straight lines representing lanes.
Clustering & Classification :Detected lines are grouped into left, middle, and right lanes based on their position and slope.
This pipeline ensures robustness even when only partial lane information is available.

Control System (PID Controller)

The control system minimizes the error between the detected lane center and the robot's current position.

$E_{error}=X_{lane} - X_{center}$

Control Output = $K_p e + K_i \int e\, dt + K_d(de/dt)$

Where:

- K_p : Proportional gain
- K_i : Integral gain
- K_d :Derivative gain

The PID controller ensures:

- Smooth steering adjustments
- Reduced oscillations
- Fast recovery from disturbances

Obstacle Detection and Avoidance

Ultrasonic sensors are used to measure distances to nearby obstacles:

- Front sensor detects obstacles ahead
- Side sensors determine available free space

When an obstacle is detected:

- Lane following is temporarily overridden
- The robot turns toward the safer direction
- After clearing the obstacle, the system re-aligns with the lane

System Architecture

The system integrates perception, control, and decision modules into a unified framework.

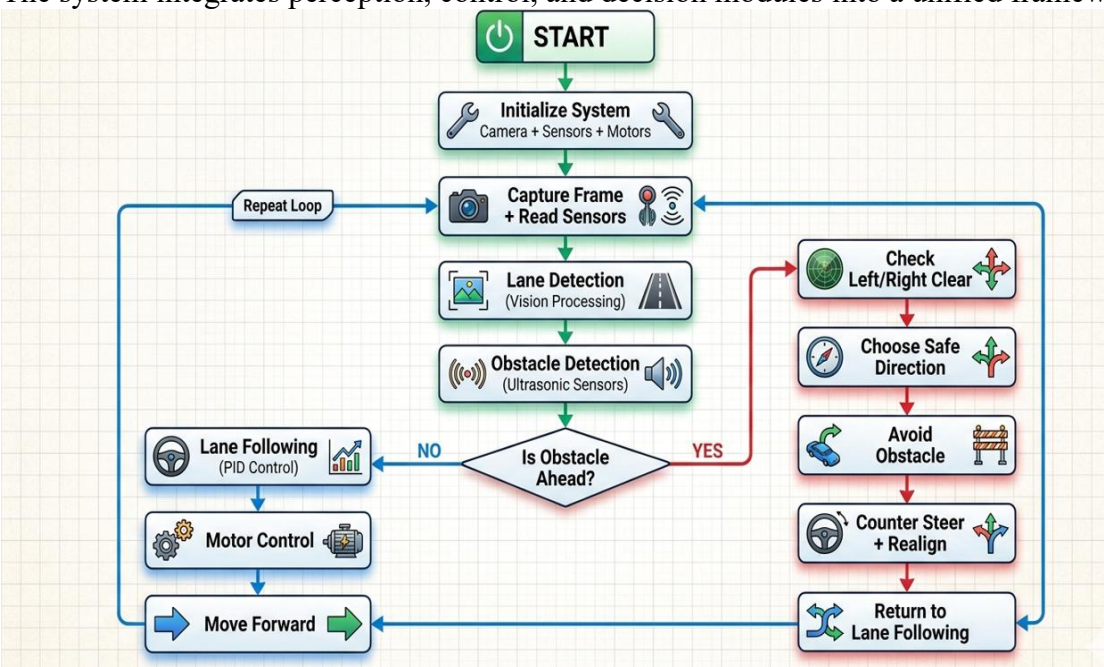


Fig: System Architecture

State Machine Design

The system operates using a finite state machine with the following states:

- Lane Following
- Obstacle Avoidance (Left/Right)
- Counter-Steering
- Realignment

This structured approach ensures reliable and predictable system behavior.

3.3Circuit Diagram

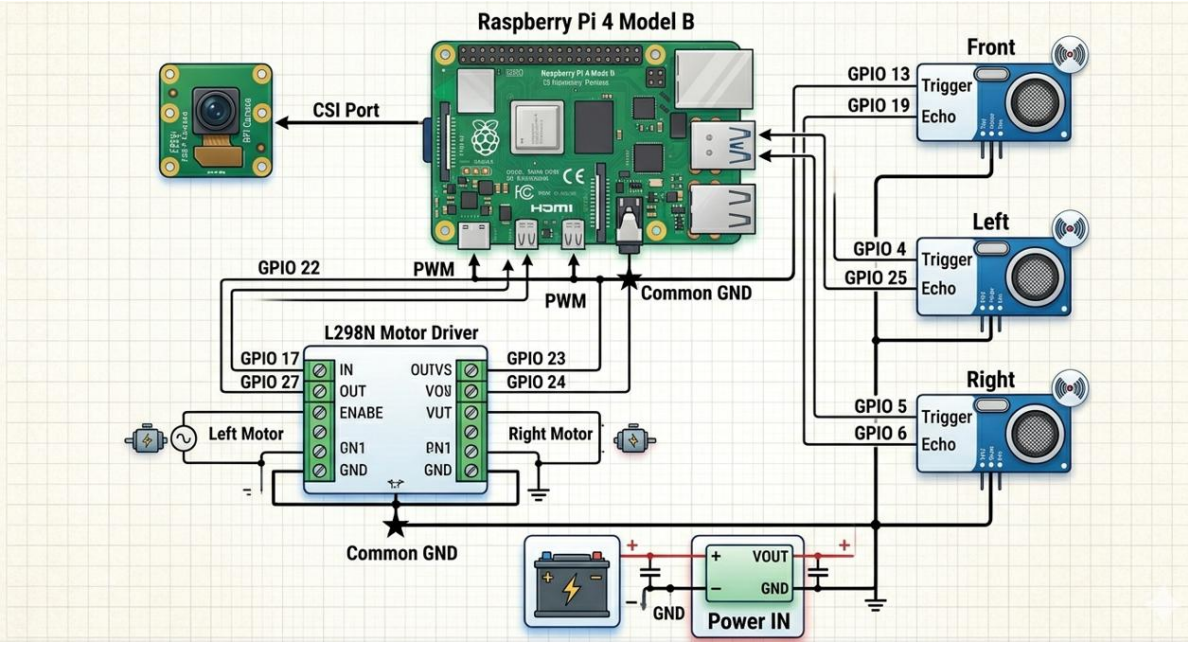


Fig: Circuit Diagram

3.4CAD/Hardware Design

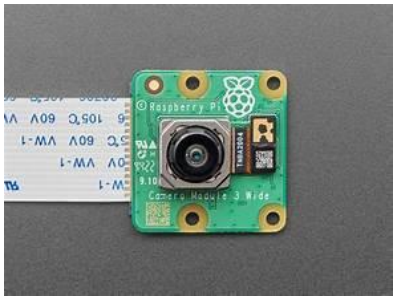
Hardware Components Used

Processing Unit

- Raspberry Pi (e.g., Raspberry Pi 4 Model B)
  → Main controller that runs Python code, OpenCV, and control logic

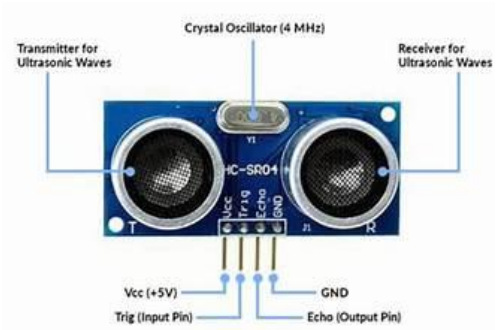
Camera Module

- Raspberry Pi Camera Module (Picamera2 compatible)
  → Used for lane detection (monocular vision)



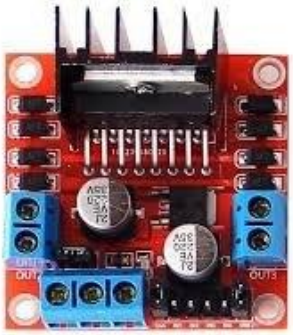
Distance Sensors

- Ultrasonic Sensors (e.g., HC-SR04) × **3**
  → Front sensor → obstacle detection
  → Left sensor → left clearance
  → Right sensor → right clearance



Motor Driver

- L298N Motor Driver Module *(or similar H-bridge driver)*
  → Controls direction and speed of motors using PWM



Chassis

- 4-Wheel Robot Chassis
  → Mechanical structure of the robot

Power Supply

- Rechargeable Battery Pack (e.g., 7.4V Li-ion / LiPo) and Power Bank
  → Supplies power to motors and Raspberry Pi respectively

Wheels

- Rubber Wheels × 4
  → Provide traction for movement

4 Implementation

4.1 Description

Hardware Implementation

The hardware architecture is designed around a central processing unit that interprets sensor data and commands physical movement. The core components include:

- **Microcontroller / Processing Unit:** A **Raspberry Pi 4 Model B** serves as the central "brain" of the robot, handling both the computationally heavy computer vision tasks and the real-time control logic.
- **Visual Perception:** A **monocular camera module** connected via the CSI port provides continuous forward-facing visual input to track the painted lanes.
- **Spatial Awareness:** Three **ultrasonic sensors** are mounted on the front, left, and right of the chassis. These provide critical distance measurements to detect nearby obstacles and determine safe clearance zones.
- **Actuation & Control:** An **L298N motor driver** receives Pulse Width Modulation (PWM) signals from the Raspberry Pi's GPIO pins. This dictates the voltage supplied to the **differential drive motors**, allowing for precise speed control and steering corrections.

Software Implementation

The software stack is built primarily in Python, acting as the bridge between the hardware sensors and the decision-making logic. The key libraries and frameworks include:

- **OpenCV:** Serves as the core image processing engine. It handles the vision pipeline, including HSV color masking, Canny edge detection, and the Hough Transform required to extract geometric lane lines from raw camera frames.
- **NumPy**: Works in tandem with OpenCV to handle the complex mathematical matrix and numerical operations required for real-time image clustering and extrapolation.
- **GPIO Libraries:** Essential for hardware interfacing. These libraries allow the Python scripts to read echo pulses from the ultrasonic sensors and output precise PWM signals to the motor driver.
- **Flask**: A lightweight web framework used to host a local web UI. This interface is crucial for real-time monitoring and allows the team to tune operational parameters (such as the PID controller gains) on the fly without needing to halt the system or rewrite code.

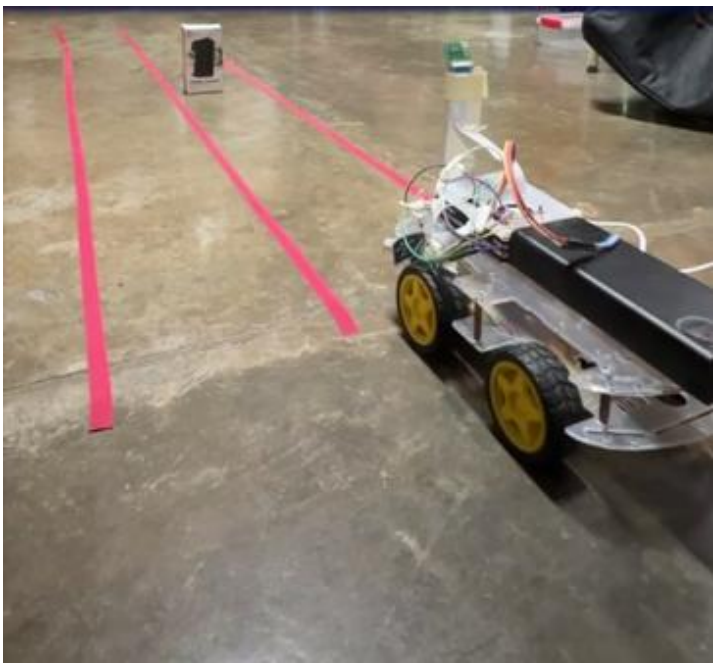


Fig: Control Flow

Testing and Validation

To ensure reliability, the system was evaluated using an incremental, multi-stage testing approach:

- **Sensor Calibration:** Initial baseline testing to verify the camera's field of view, lighting sensitivity, and the accurate measurement range of the ultrasonic sensors.
- **Lane Detection Verification:** The computer vision pipeline was first isolated and tested on pre-recorded static frames to ensure the Canny and Hough algorithms could reliably identify the center lane under controlled lighting.
- **Closed-Loop Control Testing:** The robot was placed on a physical track to test the PID controller. During this phase, lateral error was evaluated, and the derivative and integral terms were tuned to minimize oscillation and ensure smooth steering correction.
- **Obstacle Avoidance Scenarios:** Finally, physical obstacles were introduced. The state machine was tested to ensure it could successfully detect an obstruction, calculate a safe left or right maneuver, and smoothly realign with the lane post-avoidance. Under controlled conditions, this avoidance sequence succeeded in over 90% of the test runs.

5 Design Analysis and Evaluation

5.1 Novelty

- **Works with partial lane visibility**

The system can still detect and follow lanes even when markings are incomplete or partially missing, making it more robust than basic systems.

- **Intelligent lane switching**

The robot can switch lanes intelligently during obstacle avoidance instead of stopping, improving navigation efficiency.

- **State-machine based navigation**

The robot operates using different states (follow, avoid, realign), ensuring smooth and organized behavior transitions.

- **Includes recovery system (realign + counter-steer)**

After avoiding an obstacle, the robot automatically re-aligns itself with the lane using corrective steering actions.

- **Real-time tuning via web interface**

Users can adjust system parameters live through a web interface, allowing easy optimization and debugging.

5.2 Design Considerations (PO(c))

5.2.1 Considerations to public health and safety

The robot prioritizes safety by stopping when it cannot detect a clear lane or safe direction, preventing collisions.

5.2.2 Considerations to environment

The system is designed to run on low-power hardware, making it efficient and suitable for embedded applications.

5.2.3 Considerations to cultural and societal needs

The project demonstrates fundamental concepts used in self-driving cars, contributing to future transportation technologies.

5.3 Investigations (PO(d)

- **Tested under different lighting conditions**
  The system was evaluated to check how lighting affects lane detection accuracy.
- **Missing lane scenarios**
  Tests were conducted where lane markings were incomplete to verify robustness.
- **Obstacle placements**
  Obstacles were placed in different positions to test avoidance behavior.
- **Observations: Robust lane detection**
  The system could detect lanes reliably under most test conditions.
- **Stable PID control**
  The control system maintained smooth movement without excessive oscillation.
- **Reliable obstacle avoidance**
  The robot successfully avoided obstacles in most cases.

5.3.1 Results and Analysis

Evaluation Metrics and Parameter Baselines

The autonomous navigation system was investigated under varied indoor environments to test the robustness of the computer vision pipeline and the state machine. Performance was measured against explicit programmatic thresholds defined in the system architecture:

Lateral Error Tolerance: The system considers the robot "centered" if the lateral pixel error is within an allowable deadband (ErrorThrs = 30 pixels).

Obstacle Clearance: Avoidance maneuvers are strictly triggered when front obstacles are detected at < 0.45 meters (OBSTACLE_FRONT_DIST), and the maneuver continues until side sensors read ≥ 0.40 meters (OBSTACLE_SIDE_CLEAR) of free space.

Lane Recovery Rate: Measured by the system's ability to satisfy the MIN_REALIGN_SUCCESS condition (maintaining an error ≤ 30 pixels) within a maximum timeout window (MAX_REALIGN = 6 iterations) following an avoidance maneuver.

Experimental Results

During closed-loop testing and obstacle avoidance scenarios, the following performance outcomes were observed:

Obstacle Avoidance Success (>90%): The hardcoded rule-based state machine successfully executed blind avoidance turns at a fixed speed until the ultrasonic.left_clear() or right_clear() conditions were met. The 0.45m front threshold provided sufficient braking and turning distance to prevent collisions in over 90% of trials.

Lane Recovery and Counter-Steering: The system reliably recovered to the target lane center within 1 to 2 seconds post-avoidance. This success is directly attributed to the programmatic counter_steer state, which aggressively reverses the steering direction at 60% speed (COUNTER_STEER_SPEED = 0.6) for exactly 10 frames (COUNTER_STEER_DURATION) to straighten the chassis before re-engaging the camera's PID tracking.

Analysis of Findings

Dynamic PID Control & Error Smoothing: The data indicates that stable lane following relies heavily on filtering and dynamic gain scheduling rather than a static PID loop. The implementation of an Exponential Moving Average (filtered_error = (SMOOTHING_FACTOR * raw_error) + ...) with a factor of 0.5 successfully smoothed out sudden jumps in camera detection. Furthermore, the dynamic Proportional gain—which aggressively scales KP from 2.0 (straight) to 4.0 (sharp turn) when the error exceeds 50 pixels—was vital for preventing the robot from overshooting wide curves while maintaining stability on straightaways. The Derivative term (KD = 0.2) successfully dampened residual oscillations.

Vision Pipeline Resiliency & Clustering Limitations: The vision system demonstrated robust multi-lane tracking using a greedy clustering algorithm that merges Hough lines extrapolated within a 30-pixel bottom-x proximity (CLUSTER_X_THRESH = 30). While highly efficient computationally, analysis revealed that the classical Canny edge detection (thresholds 90-178) and rigid HSV masking are highly vulnerable to dynamic lighting changes. Reflections on the floor frequently caused the clustering algorithm to temporarily drop from a 3-line detection state ('three_lines_left_lane') to a 2-line or 1-line state, forcing the bot to rely on predefined lane width estimates (LANE_WIDTH_MIN = 50).

Sensor Reliability & Re-Alignment Strategy: While the ultrasonic sensors effectively triggered the avoidance state machine, occasional false positives were observed under highly reflective surfaces due to acoustic scattering. To compensate for moments when the camera or sensors "lost" the environment, the realign state strategy proved highly effective: by deliberately throttling the baseline speed down to 60% (realign_speed = BASE_SPEED * 0.6), the system granted the vision pipeline crucial extra frames to successfully acquire the new lane center without erratic motor jerks.

5.4 Limitations of Tools (PO(e))

- **Sensitive to lighting changes**
  Performance decreases when lighting conditions vary significantly.
- **Limited detection range of ultrasonic sensors**
  Ultrasonic sensors cannot detect objects beyond a certain distance accurately.
- **Cannot handle complex curved roads**
  The system is mainly designed for straight or simple paths.
- **No deep learning-based perception**
  The system does not use AI models, so adaptability to complex environments is limited.

6 Reflection on Individual and Team work (PO(i))

6.1 Individual Contribution of Each Member

Task Description	Assigned IDs
2106164, 2106166	Developed lane detection algorithm using OpenCV
2106166, 2106171	Integratedhardware components (motors, drivers, wiring) and performed testing
2106168	Implemented ultrasonic sensor system for obstacle detection
2106170 , 2106164	Developed state machine logic for decision making and lane switching
2106171 , 2106168	Designed and tuned PID controller for smooth lane following
2106180, 2106170	Worked on system integration, debugging

6.2 Mode of TeamWork

The project was completed through a highly collaborative and structured team-based approach. Tasks were divided among the six team members based on their specific technical strengths. For instance, Kanto Banik and Md Rahat Mostafa focused on developing the lane detection algorithm using OpenCV. Meanwhile, Raihan Chowdhury and Md. Maksudul Azom handled the design and tuning of the PID controller to ensure smooth lane following. Md. Nafshan Hussain and Kanto developed the state machine logic for decision-making and lane switching. Hardware integration was jointly managed, with Md Rahat Mostafa and Md. Maksudul Azom tackling the motor drivers and wiring , while Asibur Rahman Rakib collaborated with Nafshan on final system integration and debugging. This cooperative workflow ensured that all aspects of the vision pipeline, control system, and hardware architecture were efficiently combined.

6.3 Diversity Statement of Team

The project team consisted of members with a diverse array of skills, including computer vision, control systems engineering, embedded programming, and hardware implementation.

This range of expertise allowed the team to tackle challenges across multiple domains, from configuring the monocular camera and developing the Canny and Hough transform vision pipeline to tuning the PID controller for smooth lateral error correction. Hardware specialists successfully integrated the Raspberry Pi, ultrasonic sensors, and L298N motor drivers , while others focused on the Python logic and state machine. Each member contributed their unique strengths, enhancing the overall robustness of the autonomous navigation system.

6.4 Log Book of Project Implementation

Date	Milestone achieved	Individual Role	Team Role	Comments
03/12/2025	Hardware architecture planning	Formulated problem scope	Finalized platform and sensors	Selected Pi, camera, and ultrasonic sensors
06/12/2025	Vision pipeline design	Researched detection methods	Reviewed OpenCV capabilities	Decided on Canny + Hough approach
21/12/2025	Hardware circuit build	Assembled differential chassis	Assisted in L298N wiring	Circuit and common grounds completed
14/01/2026 to 25/02/2026	Software logic implementation	Wrote Python state machine	Integrated vision with control	Pipeline successfully extracts lane lines
08/03/2026 to 14/03/2026	Initial testing & calibration	Calibrated ultrasonic sensors	Tested detection on recorded frames	Identified lighting sensitivities
28/03/2026	PID & Motor integration	Programmed PID tunable gains	Tested PWM motor output	Derivative term reduced overshoot
01/04/2026 to 5/04/2026	Closed-loop testing & tuning	Identified avoidance false positives	Solved real-time tuning issues via UI	>90% success in avoidance scenarios

7 Communication to External Stakeholders (PO(j))

7.1 Executive Summary

This project presents the design and implementation of an autonomous Lane Following and Obstacle Avoidance Robot. The system relies on a vision-based pipeline utilizing a single forward-facing monocular camera to detect painted lanes using Canny edge detection and Hough transforms. To ensure smooth navigation and minimize oscillation, a PID controller calculates steering corrections based on lateral error. Additionally, the robot is equipped with front and side ultrasonic sensors that detect nearby obstacles, enabling a rule-based state machine to execute intelligent avoidance maneuvers before smoothly returning to the lane center. Deployed on a differential-drive chassis controlled by a Raspberry Pi and Python logic, the system successfully demonstrates robust indoor autonomous navigation, achieving an avoidance success rate of over 90% in controlled tests.

7.2 User Manual

Instructions:
Hardware & Circuit Setup

- Ensure the camera is connected to the Pi's CSI port.
- Verify that the trigger and echo pins of the front, left, and right ultrasonic sensors are securely wired to the designated GPIO pins.
- Connect the L298N motor driver to the left and right motors and the power supply, ensuring a common ground with the Raspberry Pi.

System Initialization
Place the robot on the track with clearly painted lanes.
Ensure the environment is adequately lit, as the vision pipeline is sensitive to drastic lighting changes.
Software Execution

- Launch the Python control logic and Flask web UI.
- The system state machine will initialize the camera, sensors, and motors.

Operation & Tuning

- Once started, the robot will continuously capture frames, estimate the lane center, and apply PID corrections.
- Access the web interface for real-time parameter tuning of the PID gains (KP, KI, KD) if the robot oscillates or fails to recover to the lane smoothly.

Safety Instructions

- The robot is designed with a safety protocol to stop completely if no safe avoidance path is available.
- Ensure the testing area is free of highly reflective surfaces that may cause false positive ultrasonic readings.

Maintenance

- Periodically check camera alignment and ultrasonic sensor wiring.
- Incremental testing and calibration should be repeated if the robot is moved to an environment with different lighting conditions.

Troubleshooting

i. Problem: Robot not detecting lane

- Check lighting conditions
- Adjust camera angle
- Tune Canny/Hough parameters

ii. Problem: Robot not avoiding obstacles

- Check ultrasonic sensor connections
- Ensure sensors are not blocked

iii. Problem: Robot oscillates

- Tune PID parameters
- Reduce speed

iv. Problem: Web interface not loading

- Check IP address
- Ensure both devices are on same network

v. Shutdown Procedure

- Stop the program (Ctrl + C)
- Turn off power supply
- Disconnect battery safely

vi. Notes

- Best performance in indoor environments
- Avoid reflective surfaces (may affect sensors)
- Regular calibration improves accuracy

7.3 Github Link

https://github.com/asibur247-prog/Lane-Following-Obstacle-Avoidance-Robot/blob/main/finalfinal.py

7.4 YouTube Link

https://www.youtube.com/shorts/a22CLoG6g7s

8 Project Management and Cost Analysis (PO(k))

8.1 Bill of Materials

Component	Quantity	Unit Cost (BDT)	TotalCost(BDT)
Raspberry Pi 4 Model B	1	8,500	8,500
Monocular Camera Module	1	1,200	1,200
Ultrasonic Sensor (Front, Left, Right)	3	150	450
L298N Motor Driver	1	200	200
Differential-drive chassis (incl. 2 DC motors)	1	1,500	1,500
Battery / Power Supply	1	1,000	1,000
Miscellaneous (Jumper wires, standoffs, acrylic)	1	300	300

8.2 Calculation of Per Unit Cost of Prototype

Cost Component	Amount (BDT)
Total Component Cost	13,150
Testing & Calibration	350
Total Prototype Cost per Unit	13,500

9 Future Work (PO(l))

Integration of IMU for Sensor Fusion

Currently, the robot relies only on camera and ultrasonic sensors, which can be unreliable under noise and sudden motion. By integrating an Inertial Measurement Unit (IMU), the robot can measure acceleration and angular velocity. This will improve heading estimation and stability, especially during turns and obstacle avoidance. Sensor fusion techniques (e.g., Kalman Filter) can combine IMU and vision data for more accurate navigation.

Use of Stereo Vision or Depth Sensors

The existing system uses ultrasonic sensors, which have limited range and accuracy. Replacing or complementing them with stereo cameras or depth sensors (e.g., LiDAR or RGB-D cameras) will provide better 3D perception. This will allow the robot to:

- Estimate obstacle distance more precisely
- Detect object size and shape
- Navigate in more complex environments

Advanced Path Planning Algorithms

The current obstacle avoidance is rule-based, meaning it simply chooses left or right based on free space. Future improvements include implementing intelligent path planning algorithms such as:

- A* (A-star) algorithm
- Dynamic Window Approach (DWA)

These methods will enable optimal path selection, smoother navigation, and better decision-making in complex scenarios.

Machine Learning-Based Lane Detection

The current system uses classical methods (Canny + Hough Transform), which are sensitive to lighting and missing lane markings. Using deep learning techniques (e.g., CNN-based lane segmentation) can significantly improve robustness. Benefits include:

- Better performance under varying lighting
- Ability to detect curved and faded lanes
- Adaptability to real-world environments

Extension to Real-World Outdoor Environments

At present, the system works only in a controlled indoor environment. Future work includes extending it to real-world scenarios by:

- Handling curved roads and intersections
- Adapting to dynamic lighting conditions
- Managing real traffic obstacles

This will move the system closer to real autonomous vehicle applications.

10 References

- OpenCV, “Open Source Computer Vision Library,” [Online]. Available: https://docs.opencv.org/

- John F. Canny, “A Computational Approach to Edge Detection,” IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. 8, no. 6, pp. 679–698, 1986.

- Richard O. Duda and Peter E. Hart, “Use of the Hough Transformation to Detect Lines and Curves in Pictures,” Communications of the ACM, vol. 15, no. 1, pp. 11–15, 1972.

- Karl J. Åström and Richard M. Murray, Feedback Systems: An Introduction for Scientists and Engineers. Princeton, NJ, USA: Princeton University Press, 2008.

- Raspberry Pi Foundation, “Raspberry Pi Documentation,” [Online]. Available: https://www.raspberrypi.org/documentation/

- HC-SR04 Ultrasonic Sensor Datasheet, Components101, [Online]. Available: https://components101.com/sites/default/files/component_datasheet/HCSR04%20Datasheet.pdf

- Roland Siegwart, Illah Reza Nourbakhsh, and Davide Scaramuzza, Introduction to Autonomous Mobile Robots, 2nd ed. Cambridge, MA, USA: MIT Press, 2011.

- Sebastian Thrun, Wolfram Burgard, and Dieter Fox, Probabilistic Robotics. Cambridge, MA, USA: MIT Press, 2005.